

Towards C++23 executors: A proposal for an initial set of algorithms

Document #: P1897R1
Date: 2019-11-13
Project: Programming Language C++ SG1
Reply-to: Lee Howes
<lwh@fb.com>

§ 1 Differences between R0 and R1

- Improve examples to be clearer, and fully expanded into function call form.
- Add reference to `range.adapter`.
- Remove `is_noexcept_sender`.
- Remove `just_error`.
- Clarified use of parameter packs of values and errors.
- Removed confusing use of `on` in addition to `via` in the final example.

§ 2 Introduction

In [P0443R11] we have included the fundamental principles described in [P1660R0], and the fundamental requirement to customize algorithms. In recent discussions we have converged to an understanding of the `submit` operation on a sender acting as a fundamental interoperation primitive, and algorithm customization giving us full flexibility to optimize, to offload and to avoid synchronization in chains of mutually compatible algorithm customizations.

As a starting point, in [P0443R11] we only include a `bulk_execute` algorithm, that satisfies the core requirement we planned with P0443 to provide scalar and bulk execution. To make the C++23 solution completely practical, we should extend the set of algorithms, however. This paper suggests an expanded initial set that enables early useful work chains. This set is intended to act as a discussion focus for us to discuss one by one, and to analyze the finer constraints of the wording to make sure we do not over-constrain the design.

In the long run we expect to have a much wider set of algorithms, potentially covering the full set in the current C++20 parallel algorithms. The precise customization of these

algorithms is open to discussion: they may be individually customized and individually defaulted, or they may be optionally individually customized but defaulted in a tree such that customizing one is known to accelerate dependencies. It is open to discussion how we achieve this and that is an independent topic, beyond the scope of this paper.

§ 2.1 Summary

Starting with [P0443R11] as a baseline we have the following customization points:

- `execute(executor, invocable) -> void`
- `submit(sender, receiver) -> void`
- `schedule(scheduler) -> sender`
- `set_done`
- `set_error`
- `set_value`

and the following Concepts:

- `executor`
- `scheduler`
- `callback_signal`
- `callback`
- `sender`

We propose immediately discussing the addition of the following algorithms:

- `just(v)`
 - returns a sender of the value `v`
- `via(s, sch)`
 - returns a sender that propagates the value or error from `s` on `sch`'s execution context
- `sync_wait(s)`
 - blocks and returns the value type of the sender, throwing on error
- `transform(s, f)`
 - returns a sender that applies `f` to the value passed by `s`, or propagates errors or cancellation
- `bulk_transform(s, f)`
 - returns a sender that applies `f` to each element in a range sent by `s`, or propagates errors or cancellation
- `handle_error(s, f)`

- returns a sender that applies `f` to an error passed by `s`, ignoring the values or cancellation

§ 2.2 Examples

§ 2.2.0.1 Simple example

A very simple example of applying a function to a propagated value and waiting for it.

```
auto just_sender =          // sender_to<int>
    just(3);

auto transform_sender =     // sender_to<float>
    transform(
        std::move(via_sender),
        [](int a){return a+0.5f;});

int result =                // value: 5
    sync_wait(std::move(error_sender));
```

In this very simple example we:

- start a chain with the value three
- apply a function to the incoming value, adding 0.5 and returning a sender of a float.
- block for the resulting value and assign the float value 3.5 to `result`.

Using operator `|` as in ranges to remove the need to pass arguments around, we can represent this as:

```
auto s = ;
float f = sync_wait(
    just(3) | transform([](int a){return a+0.5f;}));
```

§ 2.2.0.2 With exception

A simple example showing how an exception that leaks out of a transform may propagate and be thrown from `sync_wait`.

```
int result = 0;
try {
    auto just_sender = just(3);
    auto via_sender = via(std::move(just_sender), scheduler1);
    auto transform_sender = transform(
        std::move(via_sender),
        [](int a){throw 2;});
    auto skipped_transform_sender = transform(
        std::move(transform_sender),
        [](){return 3;});

    result = sync_wait(std::move(skipped_transform_sender));
} catch(int a) {
    result = a;                                // Assign 2 to
```

```
result
}
```

In this example we:

- start a chain with an int value 3
- switch the context to one owned by scheduler1
- apply a transformation to the value 3, but this transform throws an exception rather than returning a transformed value
- skip the final transform because there is an error propagating
- block for the resulting value, seeing an exception thrown instead of a value returned
- handle the exception

As before, using operator| as in ranges to remove the need to pass arguments around, we can represent this more cleanly:

```
int result = 0;
try {
    result = sync_wait(
        just(3) |
        via(scheduler1) |
        transform([](int a){throw 2;}) |
        transform([](){return 3;}));
} catch(int a) {
    result = a;
} // Assign 2 to
result
}
```

§ 2.2.0.3 Handle an exception

Very similar to the above, we can handle an error mid-stream

```
auto just_sender = just(3);
auto via_sender = via(std::move(just_sender), scheduler1);
auto transform_sender = transform(
    std::move(via_sender),
    [](int a){throw 2;});
auto skipped_transform_sender = transform(
    std::move(transform_sender),
    [](){return 3;});
auto error_handling_sender = handle_error(
    std::move(skipped_transform_sender),
    [](exception_ptr e){return just(5);});

auto result = sync_wait(std::move(error_handling_sender));
```

In this example we:

- start a chain with an int value 3
- switch the context to one owned by scheduler1

- apply a transformation to the value 3, but this transform throws an exception rather than returning a transformed value
- skip the final transform because there is an error propagating
- handle the error channel, applying an operation to an `exception_ptr` pointing to the value 2
- in handling the error we return a sender that propagates the value 5, thus recovering from the error
- block for the resulting value, assigning 5 to `result`

As before, using `operator|` as in ranges to remove the need to pass arguments around, we can represent this more cleanly:

```
auto s = ;
int result = sync_wait(
    just(3) |
    via(scheduler1) |
    transform([](float a){throw 2;}) |
    transform([](){return 3;}) |
    handle_error([](auto e){
        return just(5);}));
```

§ 3 Impact on the standard library

§ 3.1 Sender adapter objects

Taking inspiration from [range adaptors](#) define sender adapters.

Wording to be based on [range.adaptors] with the basic requirement that:

- `operator|` be overloaded for the purpose of creating pipelines over senders
- That the following are equivalent expressions:
 - `algorithm(sender, args...)`
 - `algorithm(args...)(sender)`
 - `sender | algorithm(args...)`
- that `algorithms(args...)` is a *sender adaptor closure object*
- TBD where sender adapters are declared

Details below are in loosely approximated wording and should be made consistent with [P0443R11] and the standard itself when finalized. We choose this set of algorithms as a basic set to allow a range of realistic, though still limited, compositions to be written against executors.

§ 3.2 `execution::just`

§ 3.2.1 Overview

just creates a sender that propagates a value inline to a submitted receiver.

Signature:

```
S<T...> just(T...);
```

where $S<T...>$ is an implementation-defined `typed_sender` that that sends a set of values of type $T...$ in its value channel.

[Example:

```
int r = sync_wait(just(3));  
// r==3
```

- end example]

§ 3.2.2 Wording

The expression `execution::just(t)` returns a sender, `s` wrapping the value `t`.

- If $t...$ are nothrow movable then `execution::is_noexcept_sender(s)` shall be `constexpr` and return true.
- When `execution::submit(s, r)` is called for some `r`, and r-value `s` will call `execution::set_value(r, std::move(t)...)`, inline with the caller.
- When `execution::submit(s, r)` is called for some `r`, and l-value `s` will call `execution::set_value(r, t...)`, inline with the caller.
- If moving of `t` throws, then will catch the exception and call `execution::set_error(r, e)` with the caught `exception_ptr`.

§ 3.3 execution::sync_wait

§ 3.3.1 Overview

Blocks the calling thread to wait for the passed sender to complete. Returns the value (or void if the sender carries no value), throws if an exception is propagated and throws a TBD exception type on cancellation.¹ On propagation of the `set_done()` signal, returns an empty optional.

```
T... sync_wait(S<T...>)
```

where $S<T...>$ is a sender that sends zero or one values of type $T...$ in its value channel. The existence of, and if existing the type T must be known statically and cannot be part of an overload set.

[Example:

```
int r = sync_wait(just(3));
// r==3
```

- end example]

§ 3.3.2 Wording

The name `execution::sync_wait` denotes a customization point object. The expression `execution::sync_wait(S)` for some subexpression `S` is expression-equivalent to:

- `S.sync_wait()` if that expression is valid.
- Otherwise, `sync_wait(S)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S>
void sync_wait(S) = delete;
```

and that does not include a declaration of `execution::sync_wait`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`. Waits on the synchronization primitive to block on completion of `S`.
 - If `set_value` is called on `r`, returns the passed value (or simply returns for void sender).
 - If `set_error` is called on `r`, throws the error value as an exception.
 - If `set_done` is called on `r`, throws some TBD cancellation exception type.

§ 3.4 `execution::via`

§ 3.4.1 Overview

`via` is a sender adapter that takes a sender and a scheduler and returns a sender that propagates the same value as the original, but does so on the sender's execution context.

Signature:

```
S<T...> via(S<T...>, Scheduler);
```

where `S<T>` is an implementation-defined type that is a sender that sends a value of type `T` in its value channel.

[Example:

```
static_thread_pool t{1};
int r = sync_wait(just(3) | via(t.scheduler()));
// r==3
```

§ 3.4.2 Wording

The name `execution::via` denotes a customization point object. The expression `execution::via(S, Sch)` for some subexpressions `S`, `Sch` is expression-equivalent to:

- `S.via(Sch)` if that expression is valid.
- Otherwise, `via(S, Sch)` if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class Sch>
void via(S, Sch) = delete;
```

- Otherwise constructs a receiver `r` such that when `set_value`, `set_error` or `set_done` is called on `r` the value(s) or error(s) are packaged, and a receiver `r2` constructed such that when `execution::set_value(r2)` is called, the stored value or error is transmitted and `r2` is submitted to `Sch`. If `set_error` or `set_done` is called on `r2` the error or cancellation is propagated and the packaged values ignored.
- The returned sender's value types match those of `sender1`.
- The returned sender's execution context is that of `scheduler1`.

§ 3.5 `execution::transform`

§ 3.5.1 Overview

`transform` is a sender adapter that takes a sender and an invocable and returns a sender that propagates the value resulting from calling the invocable on the value passed by the preceding sender.

Signature:

```
S<T2> transform(S<T...>, invocable<T2(T...)>);
```

where `S<T...>` and `S<T2>` are implementation-defined types that represent senders that send a value of type list `T...` or `T2` respectively in their value channels. Note that in the general case there may be many types `T...` for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
int r = sync_wait(just(3) | transform([](int v){return v+1;}));
// r==4
```

§ 3.5.2 Wording

The name `execution::transform` denotes a customization point object. The expression `execution::transform(S, F)` for some subexpressions `S` and `F` is expression-equivalent to:

- `S.transform(F)` if that expression is valid.

- Otherwise, `transform(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
    void transform(S, F) = delete;
```

and that does not include a declaration of `execution::transform`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`. When some `output_receiver` has been passed to `submit` on the returned sender.
 - If `set_value(r, Ts... ts)` is called, calls `std::invoke(F, ts...)` and passes the result `v` to `execution::set_value(output_receiver, v)`.
 - If `F` throws, catches the exception and passes it to `execution::set_error(output_receiver, e)`.
 - If `set_error(c, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.
 - If `set_done(c)` is called, calls `execution::set_done(output_receiver)`.

§ 3.6 `execution::bulk_transform`

§ 3.6.1 Overview

`bulk_transform` is a sender adapter that takes a sender of a range of values and an invocable and returns a sender that executes the invocable for each element of the input range, and propagates the range of returned values.

Signature:

```
S<range<T2>> bulk_transform(S<range<T>>, invocable<T2(T)>);
```

where `S<range<T>>` and `S<T2>` are implementation-defined types that represent senders that send a value of type `list T` or `T2` respectively in their value channels. Note that in the general case there may be many types `T` for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
std::vector<int> r = sync_wait(just(std::vector<int>{3, 4, 5}) |
bulk_transform([](int v){return v+1;}));
// r=={4, 5, 6}
```

Note: it is TBD how precisely we should represent the intermediate data types here. Intermediate vectors would require allocator support. Purely lazy ranges may be inadequate.

§ 3.6.2 Wording

The name `execution::bulk_transform` denotes a customization point object. The expression `execution::bulk_transform(S, F)` for some subexpressions `S` and `F` is expression-equivalent to:

- `S.bulk_transform(F)`, if that expression is valid.
- Otherwise, `bulk_transform(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
void bulk_transform(S, F) = delete;
```

and that does not include a declaration of `execution::bulk_transform`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.
 - If `S::value_type` does not model the concept `Range<T>` for some `T` the expression ill-formed.
 - If `set_value` is called on `r` with some parameter input applies the equivalent of `out = std::ranges::transform_view(input, F)` and passes the result output to `execution::set_value(output_receiver, v)`.
 - If `set_error(r, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.
 - If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

§ 3.7 **execution::handle_error**

§ 3.7.1 **Overview**

`handle_error` is a sender adapter that takes a sender and an invocable and returns a sender that propagates the value, error or done signal from the sender returned by the invocable.

Signature:

```
S<T2..., E2...> handle_error(S<T..., E...>,
invocable<sender<T2..., E2...>(E...)>);
```

where `S<T..., E...>` and `S<T2..., E2...>` are implementation-defined types that represent senders that send a value of type list `T...` or `T2...` respectively in their value channels and error type lists `E...` and `E2...` in their error channels. The invocable takes the error types `E...` and returns a sender over some potentially new set of types. By returning a sender the algorithm has control of error recovery as well as use cases such as logging and propagation. Note that in the general case there may be many types `E...` for a given sender, in which case the invocable may have to represent an overload set.

[Example:

```
float r = sync_wait(
    just(3) |
    transform([](int v){throw 2.0f;}) |
    handle_error([](float e){return just(e+1);}));
// r==3.0f
```

§ 3.7.2 Wording

The name `execution::handle_error` denotes a customization point object. The expression `execution::handle_error(S, F)` for some subexpressions `S` and `F` is expression-equivalent to:

- `S.handle_error(F)`, if that expression is valid.
- Otherwise, `handle_error(S, F)`, if that expression is valid with overload resolution performed in a context that includes the declaration

```
template<class S, class F>
    void handle_error(S, F) = delete;
```

and that does not include a declaration of `execution::handle_error`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes that receiver to `execution::submit(S, r)`.
 - If `set_value(r, v...)` is called, passes `v...` to `execution::set_value(output_receiver, v...)`.
 - If `set_error(r, e...)` is called, passes `e...` to `f`, resulting in a sender `s2` and passes `output_receiver` to `submit(s2, output_receiver)`.
 - If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

§ 4 Customization and example

Each of these algorithms, apart from `just`, is customizable on one or more sender implementations. This allows full optimization. For example, in the following simple work chain:

```
auto s = just(3) | // s1
    via(scheduler1) | // s2
    transform([](int a){return a+1;}) | // s3
    transform([](int a){return a*2;}) | // s4
    via(scheduler2) | // s5
    handle_error([](auto e){return just(3);}); // s6
int r = sync_wait(s);
```

The result of `s1` might be a `just_sender<int>` implemented by the standard library vendor.

`via(just_sender<int>, scheduler1)` has no customization defined, and this expression returns an `scheduler1_via_sender<int>` that is a custom type from the author of

`scheduler1`, it will call `submit` on the result of `s1`.

`s3` calls `transform(scheduler1_via_sender<int>, [](int a){return a+1;})` for which the author of `scheduler1` may have written a customization. The `scheduler1_via_sender` has stashed the value somewhere and build some work queue in the background. We do not see `submit` called at this point, it uses a behind-the-scenes implementation to schedule the work on the work queue. An `scheduler1_transform_sender<int>` is returned.

`s4` implements a very similar customization, and again does not call `submit`. There need be no synchronization in this chain.

At `s5`, however, the implementor of `scheduler2` does not know about the implementation of `scheduler1`. At this point it will call `submit` on the incoming `scheduler1_transform_sender`, forcing `scheduler1`'s sender to implement the necessary synchronization to map back from the behind-the-scenes optimal queue to something interoperable with another vendor's implementation.

`handle_error` at `s6` will be generic in terms of `submit` and not do anything special, this uses the default implementation in terms of `submit`. `sync_wait` similarly constructs a `condition_variable` and a temporary `int`, submits a receiver to `s` and waits on the `condition_variable`, blocking the calling thread.

`r` is of course the value 8 at this point assuming that neither scheduler triggered an error. If there were to be a scheduling error, then that error would propagate to `handle_error` and `r` would subsequently have the value 3.

§ 5 References

[P0443R11] 2019. A Unified Executors Proposal for C++.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0443r11.html>

[P1660R0] 2019. A Compromise Executor Design Sketch.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1660r0.pdf>

-
1. Other options include an optional return type.↩